Compañero de Reuniones de IA Empresarial STT

Introducción

Imagina que estás asistiendo a una reunión de negocios donde todas las conversaciones están siendo capturadas por una aplicación avanzada de IA. Esta aplicación no solo transcribe las discusiones con alta precisión, sino que también proporciona un resumen conciso de la reunión, enfatizando los puntos clave y las decisiones tomadas.

En nuestro proyecto, utilizaremos Whisper de OpenAI para transformar el habla en texto. A continuación, usaremos la IA de IBM Watson para resumir y encontrar puntos clave. Crearemos una aplicación con Hugging Face Gradio como interfaz de usuario.

Objetivos de Aprendizaje

Después de finalizar este laboratorio, podrás:

- Crear un script en Python para generar texto utilizando un modelo del Hugging Face Hub, identificar algunos parámetros clave que influyen en la salida del modelo y tener una comprensión básica de cómo cambiar entre diferentes modelos LLM.
- Utilizar la tecnología Whisper de OpenAI para convertir grabaciones de conferencias en texto, de manera precisa.
- Implementar la IA de IBM Watson para resumir de manera efectiva las conferencias transcritas y extraer puntos clave.
- Crear una interfaz intuitiva y fácil de usar utilizando Hugging Face Gradio,

The content of this lab is licensed under Apache 2.0

Generado por DALLE-3

Preparando el entorno

Comencemos configurando el entorno creando un entorno virtual de Python e instalando las bibliotecas necesarias, utilizando los siguientes comandos en la terminal:

```bash
pip3 install virtualenv
virtualenv my_env # create a virtual environment my_env
source my_env/bin/activate # activate my_env
```

Run

Luego, instala las bibliotecas necesarias en el entorno (esto tomará tiempo ☕ ☕):

```bash
# installing required libraries in my_env
pip install --force-reinstall "setuptools<70" transformers==4.36.0 torch==
```

Run

Toma una taza de café, tomará unos minutos.

The content of this lab is licensed under Apache 2.0

plaintext

```
     )  (
    (   ) )
     ) ( (
   _______)_
 .-'---------|
( C|/\/\/\/\/|
 '-./\/\/\/\/|
   '_________'
    '-------'
```

Necesitamos instalar `ffmpeg` para poder trabajar con archivos de audio en python.

bash

```
sudo apt update
```

Run

Luego ejecuta:

bash

```
sudo apt install ffmpeg -y
```

Run

> *Whisper de OpenAI está disponible en github. El código y los pesos del modelo de Whisper se publican bajo la Licencia MIT. Consulta LICENSE para más detalles.*

The content of this lab is licensed under Apache 2.0

Paso 1: De voz a texto

Inicialmente, queremos crear un archivo Python simple de voz a texto utilizando OpenAI Whisper.

Puedes probar el archivo de audio de muestra **Voz de muestra** [**enlace para descargar.**]

Crea y abre un archivo Python y llámalo `simple_speech2text.py` haciendo clic en el enlace de abajo:

Open simple_speech2text.py in IDE

Primero descarguemos el archivo (puedes hacerlo manualmente, luego arrástralo y suéltalo en el entorno de archivos).

python

```python
import requests

# URL of the audio file to be downloaded
url = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/

# Send a GET request to the URL to download the file
response = requests.get(url)

# Define the local file path where the audio file will be saved
audio_file_path = "downloaded_audio.mp3"

# Check if the request was successful (status code 200)
if response.status_code == 200:
    # If successful, write the content to the specified local file path
    with open(audio_file_path, "wb") as file:
        file.write(response.content)
    print("File downloaded successfully")
else:
    # If the request failed, print an error message
```

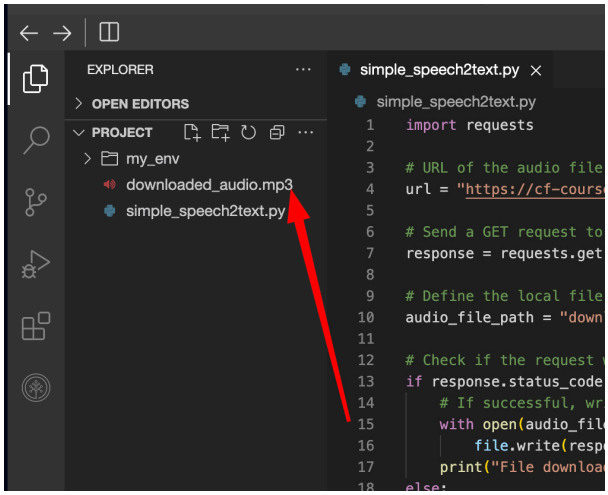
The content of this lab is licensed under Apache 2.0

Ejecuta el archivo de Python para probarlo.

```bash
python3 simple_speech2text.py
```

Run

Deberías ver el archivo de audio descargado en el explorador de archivos.



A continuación, implementa OpenAI Whisper para transcribir voz a texto.

Puedes sobrescribir el código anterior en el archivo Python.

The content of this lab is licensed under Apache 2.0

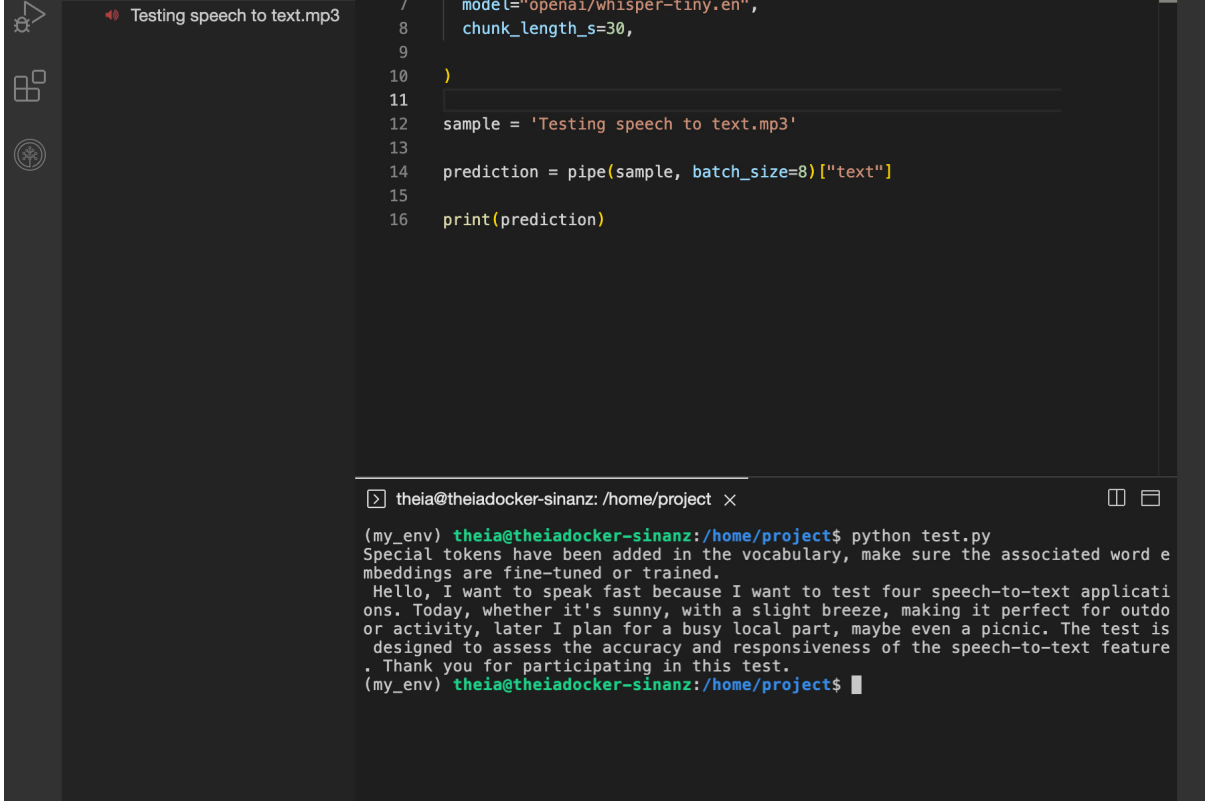
```python
import torch
from transformers import pipeline

# Initialize the speech-to-text pipeline from Hugging Face Transformers
# This uses the "openai/whisper-tiny.en" model for automatic speech recogn
# The `chunk_length_s` parameter specifies the chunk length in seconds for
pipe = pipeline(
  "automatic-speech-recognition",
  model="openai/whisper-tiny.en",
  chunk_length_s=30,
)

# Define the path to the audio file that needs to be transcribed
sample = 'downloaded_audio.mp3'

# Perform speech recognition on the audio file
# The `batch_size=8` parameter indicates how many chunks are processed at
# The result is stored in `prediction` with the key "text" containing the
prediction = pipe(sample, batch_size=8)["text"]

# Print the transcribed text to the console
print(prediction)
```

Ejecuta el archivo de Python y obtendrás la salida.

```bash
python3 simple_speech2text.py
```

Run

The content of this lab is licensed under Apache 2.0



En el siguiente paso, utilizaremos `Gradio` para crear la interfaz de nuestra aplicación.

The content of this lab is licensed under Apache 2.0

Interfaz de Gradio

Creando una demostración simple

A través de este proyecto, crearemos diferentes aplicaciones de LLM con la interfaz de Gradio. Familiaricémonos con Gradio creando una aplicación simple:

Aún en el directorio `project`, crea un archivo de Python y nómbralo `hello.py`.

Abre `hello.py`, pega el siguiente código de Python y guarda el archivo.

python

```python
import gradio as gr

def greet(name):
    return "Hello " + name + "!"

demo = gr.Interface(fn=greet, inputs="text", outputs="text")

demo.launch(server_name="0.0.0.0", server_port= 7860)
```

El código anterior crea una **gradio.Interface** llamada `demo`. Envuelve la función `greet` con una interfaz de usuario simple de texto a texto con la que puedes interactuar.

La clase **gradio.Interface** se inicializa con 3 parámetros obligatorios:

- fn: la función a la que se le aplicará la interfaz
- inputs: qué componente(s) usar para la entrada (por ejemplo, "texto", "imagen" o "audio")
- outputs: qué componente(s) usar para la salida (por ejemplo, "texto", "imagen" o "etiqueta")

La última línea `demo.launch()` lanza un servidor para servir nuestro `demo`.

The content of this lab is licensed under Apache 2.0

Lanzando la aplicación de demostración

Ahora vuelve a la terminal y asegúrate de que el nombre del entorno virtual `my_env` se muestre al principio de la línea.

A continuación, ejecuta el siguiente comando para ejecutar el script de Python.

```bash
python3 hello.py
```

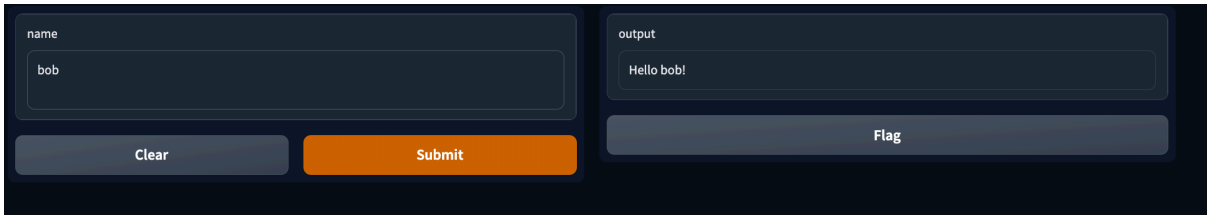
Run

Como el código de Python se sirve desde un host local, haz clic en el botón de abajo y podrás ver la aplicación simple que acabamos de crear. ¡Siéntete libre de experimentar con la entrada y salida de la aplicación web!

Haz clic aquí para ver la aplicación:

Aplicación web

Deberías ver lo siguiente, aquí ingresamos el nombre Bob:



Si terminas de jugar con la aplicación y deseas salir, **presiona `Ctrl+c` en la terminal y cierra la pestaña de la aplicación**.

Si deseas aprender un poco más sobre la personalización en Gradio, estás invitado a realizar el proyecto guiado llamado **Dale vida a tu modelo de Machine Learning con Gradio**. Puedes encontrarlo en **Cursos y Proyectos** en cognitiveclass.ai!

Para el resto de este proyecto, utilizaremos Gradio como interfaz para aplicaciones LLM.

The content of this lab is licensed under Apache 2.0

The content of this lab is licensed under Apache 2.0

Paso 2: Creando una aplicación de transcripción de audio

Crea un nuevo archivo de python `speech2text_app.py`

Open speech2text_app.py in IDE

Ejercicio: Completa la función `transcript_audio`.

Desde el paso 1: completa las partes que faltan en la función `transcript_audio`.

python

```python
import torch
from transformers import pipeline
import gradio as gr

# Function to transcribe audio using the OpenAI Whisper model
def transcript_audio(audio_file):
    # Initialize the speech recognition pipeline
    pipe = #-----> Fill here <----

    # Transcribe the audio file and return the result
    result = #-----> Fill here <----
    return result

# Set up Gradio interface
audio_input = gr.Audio(sources="upload", type="filepath")  # Audio input
output_text = gr.Textbox()  # Text output

# Create the Gradio interface with the function, inputs, and outputs
iface = gr.Interface(fn=transcript_audio,
                     inputs=audio_input, outputs=output_text,
                     title="Audio Transcription App",
                     description="Upload the audio file")

# Launch the Gradio app
iface.launch(server_name="0.0.0.0", server_port=7860)
```

The content of this lab is licensed under Apache 2.0

▶ Haz clic aquí para la respuesta

Luego, ejecuta tu aplicación:

```bash
python3 speech2text_app.py
```

Run

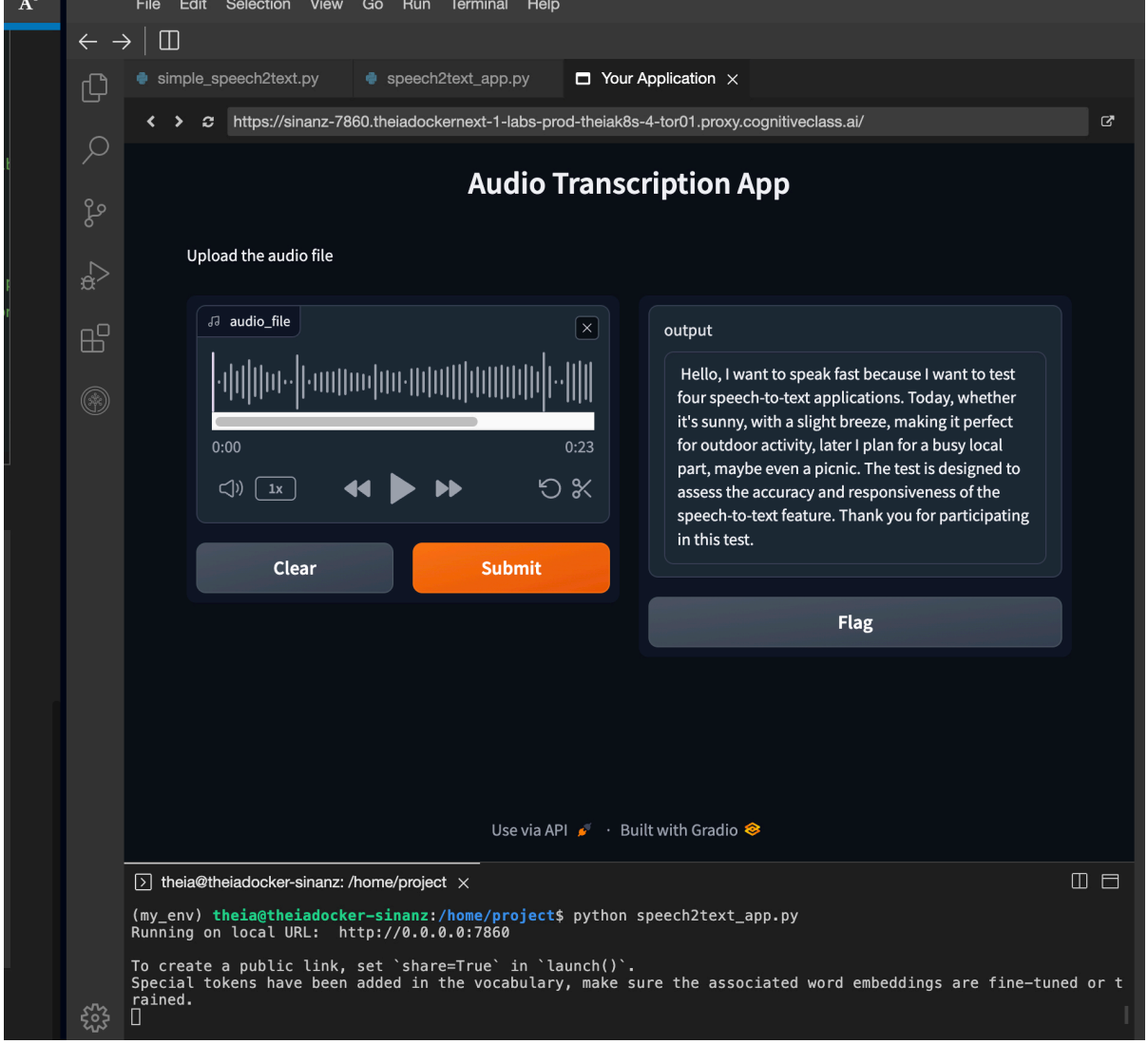
Y comienza la aplicación:

Web application

Puedes descargar el archivo de audio de muestra que hemos proporcionado haciendo clic derecho sobre él en el explorador de archivos y seleccionando "Descargar". Una vez descargado, puedes subir este archivo a la aplicación. Alternativamente, siéntete libre de elegir y subir cualquier archivo de audio MP3 de tu computadora local.

El resultado será:

The content of this lab is licensed under Apache 2.0



Presiona Ctrl + C para detener la aplicación.

The content of this lab is licensed under Apache 2.0

Paso 3: Integración de LLM: Usando Llama 3 en WatsonX como LLM

Ejecución de LLM simple

Comencemos generando texto con LLMs. Crea un archivo de Python y nómbralo `simple_llm.py`. Puedes proceder haciendo clic en el enlace a continuación o haciendo referencia a la imagen que acompaña.

Open simple_llm.py in IDE

En caso de que desees usar Llama 3 como una instancia de LLM, puedes seguir las instrucciones a continuación:

> *IBM WatsonX utiliza varios modelos de lenguaje, incluido Llama 3 de Meta, que es actualmente el modelo de lenguaje de código abierto más potente.*

Así es como funciona el código:

1. **Configuración de credenciales**: Las credenciales necesarias para acceder a los servicios de IBM están preestablecidas por el equipo de Skills Network, por lo que no tienes que preocuparte por configurarlas tú mismo.

2. **Especificación de parámetros**: El código define luego parámetros específicos para el modelo de lenguaje. 'MAX_NEW_TOKENS' establece el límite en el número de palabras que el modelo puede generar de una vez. 'TEMPERATURE' ajusta cuán creativo o predecible es el texto generado.

3. **Configuración del modelo Llama 3**: A continuación, se configura el modelo LLAMA3 utilizando un ID de modelo, las credenciales proporcionadas, los parámetros elegidos y un ID de proyecto.

The content of this lab is licensed under Apache 2.0

4. **Creación de un objeto para Llama 3**: El código crea un objeto llamado llm, que se utiliza para interactuar con el modelo Llama 3. Se crea un objeto de modelo, LLAMA3_model, utilizando la clase Model, que se inicializa con un ID de modelo específico, credenciales, parámetros e ID de proyecto. Luego, se crea una instancia de WatsonxLLM con LLAMA3_model como argumento, inicializando el objeto del hub de modelo de lenguaje llm.

5. **Generación e impresión de la respuesta**: Finalmente, se utiliza 'llm' para generar una respuesta a la pregunta, "¿Cómo leer un libro de manera efectiva?" La respuesta se imprime a continuación.

python

```python
from ibm_watson_machine_learning.foundation_models import Model
from ibm_watson_machine_learning.foundation_models.extensions.langchain im
from ibm_watson_machine_learning.metanames import GenTextParamsMetaNames a

my_credentials = {
    "url"    : "https://us-south.ml.cloud.ibm.com"
}

params = {
        GenParams.MAX_NEW_TOKENS: 700, # The maximum number of tokens that
        GenParams.TEMPERATURE: 0.1,   # A parameter that controls the rand
    }

LLAMA2_model = Model(
        model_id= 'meta-llama/llama-4-maverick-17b-128e-instruct-fp8',
        credentials=my_credentials,
        params=params,
        project_id="skills-network",
        )

llm = WatsonxLLM(LLAMA2_model)

print(llm("How to read a book effectively?"))
```

Puedes ejecutar este script en la terminal utilizando el siguiente comando:

bash

```bash
python3 simple_llm.py
```

Run

The content of this lab is licensed under Apache 2.0

Al ejecutar el script, deberías ver el texto generado en tu terminal, como se muestra a continuación:

```
Reading is one of the most efficient ways to gain knowledge and expand your mind. However, not all reading is created equal. Here are some tips to help you read a book effectively:

1. Set goals: Before you start reading, set specific goals for what you want to achieve. Do you want to learn a new skill or gain a deeper understanding of a particular subject? Having clear goals in mind will help
2. Choose the right book: Not all books are created equal. Choose a book that aligns with your goals and interests. Look for books that have received positive reviews and are written by experts in their field.
3. Create a reading schedule: Set aside dedicated time each day or week to read. Consistency is key to making progress and retaining information.
4. Take notes: Taking notes while reading can help you retain information and engage with the material on a deeper level. Write down key points, questions, and insights that come to mind as you read.
5. Summarize the material: After finishing a chapter or section, summarize the material in your own words. This will help you understand the material better and
```

Puedes ver cómo watsonx Llama 2 proporciona una buena respuesta.

The content of this lab is licensed under Apache 2.0

Paso 4: Juntar todo

Crea un nuevo archivo de Python y llámalo `speech_analyzer.py`

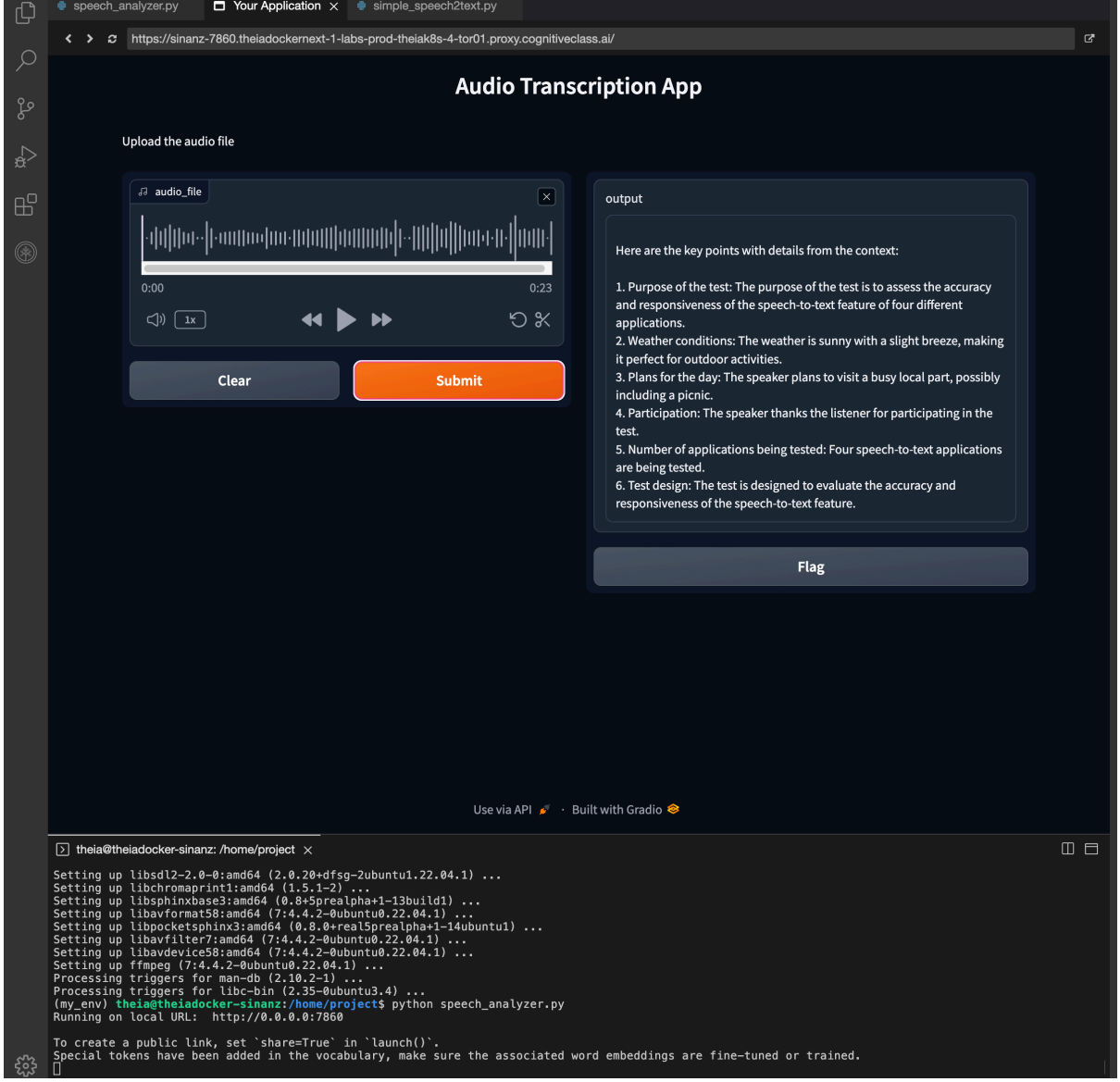
Open speech_analyzer.py in IDE

En este ejercicio, configuraremos una instancia de modelo de lenguaje (LLM), que podría ser IBM WatsonxLLM, HuggingFaceHub o un modelo de OpenAI. Luego, estableceremos una plantilla de aviso. Estas plantillas son guías estructuradas para generar avisos para modelos de lenguaje, ayudando a la organización de la salida (más información en langchain prompt template.

A continuación, desarrollaremos una función de transcripción que emplea el modelo Whisper de OpenAI para convertir el habla en texto. Esta función toma un archivo de audio subido a través de una interfaz de aplicación Gradio (preferiblemente en formato .mp3). El texto transcrito se alimenta luego a un LLMChain, que integra el texto con la plantilla de aviso y lo envía al LLM elegido. La salida final del LLM se muestra en el cuadro de texto de salida de la aplicación Gradio.

La salida debería verse así:

The content of this lab is licensed under Apache 2.0



Observa cómo el LLM corrigió un pequeño error cometido por el modelo de habla a texto, resultando en una salida coherente y precisa.

The content of this lab is licensed under Apache 2.0

Ejercicio: Completa las partes que faltan:

python

```python
import torch
import os
import gradio as gr
#from langchain.llms import OpenAI
from langchain.llms import HuggingFaceHub
from transformers import pipeline
from langchain.prompts import PromptTemplate
from langchain.chains import LLMChain
from ibm_watson_machine_learning.foundation_models.extensions.langchain im
from ibm_watson_machine_learning.foundation_models.utils.enums import Decc
from ibm_watson_machine_learning.metanames import GenTextParamsMetaNames a
from ibm_watson_machine_learning.foundation_models import Model

#######------------ LLM------------####

# initiate LLM instance, this can be IBM WatsonX, huggingface, or OpenAI i

llm = ###---> write your code here

#######------------ Prompt Template------------####

# This template is structured based on LLAMA2. If you are using other LLMs
temp = """
<s><<SYS>>
List the key points with details from the context:
[INST] The context : {context} [/INST]
<</SYS>>
"""
# here is the simplified version of the prompt template
# temp = """
# List the key points with details from the context:
# The context : {context}
# """

pt = PromptTemplate(
    input_variables=["context"],
    template= temp)

prompt_to_LLAMA2 = LLMChain(llm=llm, prompt=pt)

#######------------ Speech2text------------####

def transcript_audio(audio_file):
    # Initialize the speech recognition pipeline
```

The content of this lab is licensed under Apache 2.0

```
    # Transcribe the audio file and return the result
    transcript_txt = pipe(audio_file, batch_size=8)["text"]
    # run the chain to merge transcript text with the template and send it
    result = prompt_to_LLAMA2.run(transcript_txt)

    return result

#######------------- Gradio------------####

audio_input = gr.Audio(sources="upload", type="filepath")
output_text = gr.Textbox()

# Create the Gradio interface with the function, inputs, and outputs
iface = #---> write code here

iface.launch(server_name="0.0.0.0", server_port=7860)
```

▶ **Haz clic aquí para la respuesta**

Ejecuta tu código:

```bash
python3 speech_analyzer.py
```

Si no hay error, ejecuta la aplicación web:

Aplicación web

The content of this lab is licensed under Apache 2.0

Conclusión

¡Felicidades por completar este proyecto! Ahora has sentado una base sólida para aprovechar potentes Modelos de Lenguaje (LLMs) para tareas de generación de texto a partir de voz. Aquí tienes un breve resumen de lo que has logrado:

- Generación de texto con LLM: Has creado un script en Python para generar texto utilizando un modelo del Hugging Face Hub, aprendido sobre algunos parámetros clave que influyen en la salida del modelo y tienes un entendimiento básico de cómo cambiar entre diferentes modelos de LLM.
- Conversión de voz a texto: Utiliza la tecnología Whisper de OpenAI para convertir grabaciones de conferencias en texto de manera precisa.
- Resumen de contenido: Implementa la IA de IBM Watson para resumir de manera efectiva las conferencias transcritas y extraer puntos clave.
- Desarrollo de interfaz de usuario: Crea una interfaz intuitiva y fácil de usar utilizando Hugging Face Gradio, asegurando la facilidad de uso para estudiantes y educadores.

Author(s)

Sina Nazeri

© IBM Corporation. Todos los derechos reservados.

Compañero de Reuniones de IA

The content of this lab is licensed under Apache 2.0

Empresarial STT

Introducción

Imagina que estás asistiendo a una reunión de negocios donde todas las conversaciones están siendo capturadas por una aplicación avanzada de IA. Esta aplicación no solo transcribe las discusiones con alta precisión, sino que también proporciona un resumen conciso de la reunión, enfatizando los puntos clave y las decisiones tomadas.

En nuestro proyecto, utilizaremos Whisper de OpenAI para transformar el habla en texto. A continuación, usaremos la IA de IBM Watson para resumir y encontrar puntos clave. Crearemos una aplicación con Hugging Face Gradio como interfaz de usuario.

Objetivos de Aprendizaje

Después de finalizar este laboratorio, podrás:

- Crear un script en Python para generar texto utilizando un modelo del Hugging Face Hub, identificar algunos parámetros clave que influyen en la salida del modelo y tener una comprensión básica de cómo cambiar entre diferentes modelos LLM.
- Utilizar la tecnología Whisper de OpenAI para convertir grabaciones de conferencias en texto, de manera precisa.
- Implementar la IA de IBM Watson para resumir de manera efectiva las conferencias transcritas y extraer puntos clave.
- Crear una interfaz intuitiva y fácil de usar utilizando Hugging Face Gradio, asegurando la facilidad de uso para estudiantes y educadores.

The content of this lab is licensed under Apache 2.0

Generado por DALLE-3

Preparando el entorno

Comencemos configurando el entorno creando un entorno virtual de Python e instalando las bibliotecas necesarias, utilizando los siguientes comandos en la terminal:

```bash
pip3 install virtualenv
virtualenv my_env # create a virtual environment my_env
source my_env/bin/activate # activate my_env
```

Run

Luego, instala las bibliotecas necesarias en el entorno (esto tomará tiempo ☕ ☕):

```bash
# installing required libraries in my_env
pip install --force-reinstall "setuptools<70" transformers==4.36.0 torch==
```

Run

Toma una taza de café, tomará unos minutos.

The content of this lab is licensed under Apache 2.0

plaintext

```
     )  (
    (   ) )
     ) ( (
   _______)_
 .-'---------|
( C|/\/\/\/\/|
 '-./\/\/\/\/|
   '_________'
    '-------'
```

Necesitamos instalar `ffmpeg` para poder trabajar con archivos de audio en python.

bash

```
sudo apt update
```

Run

Luego ejecuta:

bash

```
sudo apt install ffmpeg -y
```

Run

> *Whisper de OpenAI está disponible en github. El código y los pesos del modelo de Whisper se publican bajo la Licencia MIT. Consulta LICENSE para más detalles.*

Paso 1: De voz a texto →

The content of this lab is licensed under Apache 2.0